

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN



Môn học: Nhập môn Trí tuệ Nhân tạo

Lớp: 24CLC03

BÁO CÁO ĐỒ ÁN LAB 1

FloodRoute HCMC

Sinh viên

Ngô Quang Thắng 23127473

Vũ Lê Trọng Văn 20127095

Nguyễn Duy Khang 23127202

Lưu Huy Minh Quang 23127016

Giảng viên

TS. Nguyễn Văn Tân

Contents

1	Giới thiệu nhóm và phạm vi đề án	1
2	Bối cảnh giao thông Việt Nam	1
3	Mô hình hóa bài toán	2
4	Dataset	2
5	Nguyên lý các thuật toán	3
6	Heuristic A*	4
7	Kiến trúc hệ thống và luồng thực thi	4
8	Thực nghiệm và so sánh	5
9	Tối ưu tuyến giao hàng nhiều điểm	7
10	Cài đặt, giới hạn và hướng phát triển	11

1 Giới thiệu nhóm và phạm vi đồ án

1.1 Bối cảnh và mục tiêu

FloodRoute HCMC là đồ án Lab 1 của nhóm em trong môn Nhập môn Trí tuệ Nhân tạo. Mục tiêu của nhóm là mô hình hóa bài toán tìm đường giao hàng trên graph có hướng, cài đặt các thuật toán tìm kiếm và so sánh kết quả theo composite cost. Nhóm em dùng UCS làm mốc để kiểm tra A* và dùng các scenario traffic/flood để minh họa ảnh hưởng của môi trường.

1.2 Thành viên và phân công

Thành viên	Mã số	Phạm vi chính
Ngô Quang Thắng	23127473	Mô hình graph, cost engine, tích hợp backend
Vũ Lê Trọng Văn	20127095	Các thuật toán tìm kiếm và trace frontier
Nguyễn Duy Khang	23127202	Dataset, scenario và kiểm thử
Lưu Huy Minh Quang	23127016	Frontend, bản đồ và báo cáo

1.3 Phạm vi

Trong báo cáo, nhóm em sử dụng bản đồ chính gồm 65 landmark và 178 directed road-path edges. Các scenario peak/rain trong phần kiểm chứng dùng graph nhỏ có nhãn SIMULATED, không đại diện cho traffic thời gian thực.

2 Bối cảnh giao thông Việt Nam

2.1 Bài toán thực tế

TP. Hồ Chí Minh, đặc biệt là khu vực TP. Thủ Đức, thường có mật độ giao thông cao và nhiều tuyến đường một chiều. Vì vậy, tuyến đường ngắn nhất về khoảng cách chưa chắc là tuyến nhanh hoặc an toàn nhất khi traffic tăng, đường bị đóng hoặc xuất hiện ngập cục bộ.

2.2 Các yếu tố ảnh hưởng

Mỗi đoạn đường trong mô hình có hướng đi, thời gian khi đường thông thoáng, mức phạt do traffic, nguy cơ ngập và trạng thái đóng/mở. Nếu một đoạn đường bị đóng thì thuật toán không được phép đi qua đoạn đó. Traffic và flood được xem là các yếu tố làm thay đổi chi phí tùy theo từng scenario.

2.3 Kịch bản shipper

Trong kịch bản đặt ra, shipper xuất phát từ depot và giao hàng đến một hoặc nhiều landmark. Với một cặp start-goal, nhóm em cần tìm tuyến có tổng chi phí nhỏ nhất. Khi có nhiều điểm giao hàng, ngoài việc tìm đường giữa hai điểm còn phải quyết định thứ tự ghé thăm. Phần này được xem như một bài toán TSP delivery và trình bày ở Chương 9.

2.4 Vì sao không chỉ tối ưu khoảng cách

Khoảng cách Haversine chỉ phản ánh khoảng cách địa lý, không thể hiện hướng đường, mức độ tắc nghẽn hay nguy cơ ngập. Vì vậy, nhóm em tối ưu theo tổng composite cost. Heuristic A* chỉ dùng một lower bound an toàn để vẫn giữ được tính tối ưu của thuật toán.

3 Mô hình hóa bài toán

3.1 Graph có hướng

Nhóm em biểu diễn bản đồ dưới dạng graph có hướng $G = (V, E)$, trong đó node là các landmark và edge là các đoạn đường có thể đi qua. Mỗi node có mã, tên và tọa độ địa lý. Mỗi edge lưu node đầu, node cuối cùng các thông tin liên quan đến đoạn đường. Trong quá trình tìm kiếm, trạng thái của một node gồm chi phí tạm thời $g(n)$, node cha và trạng thái đã được khám phá hay chưa.

3.2 Thuộc tính edge

Một edge có các thuộc tính: distance, free-flow time, traffic penalty, flood risk, direction và closure status. Transition chỉ được phép trên edge không đóng. Graph được giữ bất biến trong suốt quá trình chạy. Các thuật toán chỉ đọc graph, không tự ý thay đổi dữ liệu gốc và sử dụng quy tắc tie-break cố định để kết quả có thể lặp lại.

3.3 Composite cost

Với scenario s , chi phí của một edge được tính bởi

$$\text{Cost}_s(e) = w_d d(e) + w_t t(e) + w_c c_s(e) + w_f r_s(e).$$

Trong đó d là distance, t là free-flow time, c_s là traffic penalty và r_s là flood risk. Các weight trong đồ án là hệ số ưu tiên dùng cho prototype và được chọn theo hướng mô phỏng/empirical. Đây không phải là phần trăm đóng góp thực tế, vì các thành phần có đơn vị và thang đo khác nhau.

3.4 Ba cost profiles

Profile	(w_d, w_t, w_c, w_f)
BALANCED	(0.25, 0.30, 0.20, 0.25)
PEAK_TRAFFIC	(0.15, 0.25, 0.45, 0.15)
RAIN_SAFE	(0.10, 0.25, 0.15, 0.50)

Các profile có thể được thay bằng API custom weights nếu có validation; closure vẫn là hard constraint.

4 Dataset

4.1 Nguồn và quy mô dữ liệu

Trong phiên bản hiện tại, ứng dụng sử dụng release `thu_duc_landmarks_v1.0.0`, gồm 65 landmark và 178 directed road-path edges tại khu vực Thủ Đức. Thông tin vị trí và đường được xây dựng từ OSM/UTraffic; một số thuộc tính traffic/flood được bổ sung để chạy các scenario thử nghiệm.

4.2 Các thuộc tính và giả định

Các thuộc tính chính của edge được nhóm em sử dụng như sau:

Thuộc tính	Ý nghĩa	Nguồn/nhãn
distance	Khoảng cách của edge	DERIVED
free-flow time	Thời gian khi đường thông thoáng	SOURCE_BACKED/DERIVED
congestion	Mức độ traffic và traffic penalty	SIMULATED hoặc dữ liệu lịch sử
flood risk	Nguy cơ ngập của edge	SIMULATED/ASSUMPTION
direction	Hướng đi chuyển của đường	SOURCE_BACKED/DERIVED
closure status	Edge có được phép đi qua hay không	Scenario

Dataset chính được dùng trong giao diện và benchmark bản đồ 65 node. Các giá trị traffic/flood trong fixture kiểm chứng được ghi rõ là SIMULATED; vì vậy kết quả dùng để đánh giá thuật toán, không phải hướng dẫn giao thông thực tế.

5 Nguyên lý các thuật toán

Trong đồ án, nhóm em cài đặt và so sánh các thuật toán sau. Bảng dưới đây tóm tắt điều kiện áp dụng trên graph hữu hạn và cost không âm; riêng các thuật toán tour được đánh giá với số điểm giao hàng nhỏ.

Thuật toán	Quy tắc chọn node	Thời gian	Bộ nhớ	Đầy đủ?	Tối ưu?
BFS	Mở rộng theo từng lớp	$O(V + E)$	$O(V)$	Có*	Có*
DFS	Đi sâu một nhánh trước	$O(V + E)$	$O(V)$	Có*	Không
UCS/Dijkstra	Chọn g nhỏ nhất	$O((V + E) \log V)$	$O(V)$	Có	Có
A*	Chọn $f = g + h$ nhỏ nhất	Phụ thuộc h	$O(V)$	Có	Có**
Greedy Best-First	Chọn h nhỏ nhất	Phụ thuộc h	$O(V)$	Có*	Không
Bidirectional	Tìm từ hai đầu	Thường giảm so với UCS	$O(b^{d/2})$	Có*	Có***

Các dấu sao trong bảng cần được hiểu theo điều kiện cụ thể. BFS tối ưu khi mọi edge có cùng cost, nên trong bài này chủ yếu dùng để so sánh. DFS có thể tìm được đường trên graph hữu hạn nhưng không đảm bảo đó là đường rẻ nhất. UCS được nhóm em dùng làm mốc tham chiếu vì tối ưu với mọi edge cost không âm. A* có cùng bảo đảm với UCS khi heuristic admissible và consistent; đây là lý do nhóm em kiểm tra A* với UCS thay vì chỉ nhìn thời gian chạy. Greedy thường nhanh nhưng không thể dùng để kết luận nghiệm tối ưu. Bidirectional cần điều kiện dừng đúng và cách xử lý graph có hướng rõ ràng.

Với bài toán đi qua nhiều điểm, Held–Karp là dynamic programming chính xác với độ phức tạp $O(n^2 2^n)$, còn Nearest Neighbor là heuristic tham lam khoảng $O(n^2)$ và không bảo đảm tối ưu. Vì vậy nhóm em dùng Held–Karp làm mốc khi số stop nhỏ và dùng Nearest Neighbor để minh họa sự đánh đổi giữa chất lượng tour và thời gian chạy.

Thuật toán tour	Cách chọn	Thời gian	Tối ưu?
Held–Karp	Lưu nghiệm tốt nhất theo tập stop đã đi qua	$O(n^2 2^n)$	Có
Nearest Neighbor	Chọn stop tiếp theo có pairwise cost nhỏ nhất	$O(n^2)$	Không

- (*) Đầy đủ trên graph hữu hạn khi cách cài đặt có kiểm tra trạng thái đã thăm và không bị nhánh vô hạn.
- (**) A* tối ưu trong report vì dùng lower-bound heuristic của Chương 6; dùng heuristic tùy ý thì không còn kết luận này.
- (***) Bidirectional tối ưu khi hai hướng dùng cost tương thích và điều kiện dừng được chứng minh đúng.

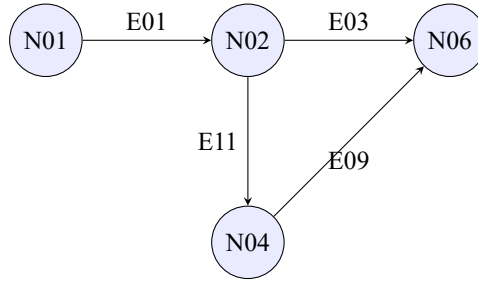


Figure 1: Toy graph minh họa hai phương án từ N01 đến N06. Thuật toán tìm kiếm đánh giá route theo tổng composite cost, không chỉ theo số cạnh.

6 Heuristic A*

6.1 Định nghĩa heuristic

Với mỗi scenario s , nhóm em tính ρ_s một lần trước khi bắt đầu tìm kiếm:

$$\rho_s = \min_{\substack{e \in E, e \text{ không đóng} \\ \text{Haversine}(e) > 10^{-6} \text{ km}}} \frac{\text{Cost}_s(e)}{\text{Haversine}(e)}.$$

Sau đó:

$$h_s(n) = \rho_s \cdot \text{Haversine}(n, \text{goal}), \quad f(n) = g(n) + h_s(n).$$

Edge đã đóng không được đưa vào phép tính ρ_s . Edge có khoảng cách Haversine gần 0 cũng được bỏ qua để tránh chia cho số quá nhỏ. Nếu không còn edge hợp lệ thì $\rho_s = 0$; node thiếu tọa độ có $h(n) = 0$.

6.2 Admissibility và consistency

Vì ρ_s không lớn hơn tỉ số giữa cost và khoảng cách của mọi edge hợp lệ, đồng thời Haversine thỏa bất đẳng thức tam giác, ta có:

$$h_s(n) \leq \text{cost}(\text{một đường đi từ } n \text{ đến goal}).$$

Do đó $h_s(n) \geq 0$ và heuristic là admissible. Với một edge $e = (u, v)$, ta cũng có:

$$h_s(u) \leq \text{Cost}_s(e) + h_s(v),$$

nên heuristic là consistent. Khi edge cost không âm, A* trả về route có composite cost tối ưu, giống như UCS.

6.3 Ảnh hưởng của scenario

Scenario được giữ cố định trong một lần tìm kiếm nên cost edge và ρ_s không thay đổi giữa các bước mở rộng node. Khi traffic, flood hoặc weight thay đổi, cần tính lại ρ_s . Heuristic này được thiết kế cho composite cost đang chọn, không phải để tối ưu riêng khoảng cách hoặc thời gian.

7 Kiến trúc hệ thống và luồng thực thi

7.1 Kiến trúc

Nhóm em chia chương trình thành các phần tương đối độc lập. Graph contract và các thuật toán nằm ở phần core, không phụ thuộc trực tiếp vào FastAPI. ScenarioCostEngine chịu trách nhiệm tính chi phí edge và heuristic lower-bound. SearchService chạy thuật toán tìm kiếm và lưu trace. PairwiseMatrix tạo ma trận chi phí giữa các

điểm, sau đó TourOptimizerService dùng ma trận này cho Held-Karp hoặc Nearest Neighbor. FastAPI cung cấp API, còn React/MapLibre hiển thị bản đồ và kết quả.

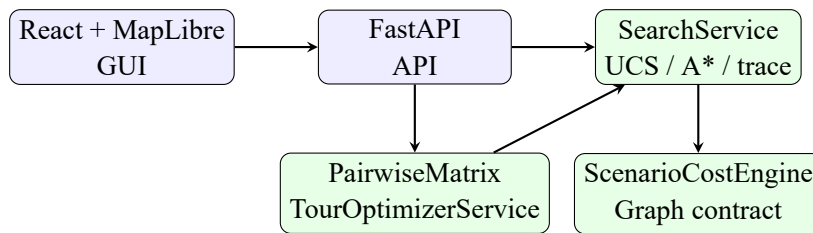


Figure 2: Sơ đồ các thành phần chính của FloodRoute HCMC.

7.2 Các luồng chính

1. Tìm đường giữa hai điểm: nạp graph, giữ scenario cố định, tính ρ_s , chạy thuật toán và trả về path/cost.
2. So sánh thuật toán: chạy nhiều thuật toán trên cùng graph, start, goal và scenario.
3. Theo dõi trace: lưu frontier, current và closed để giao diện có thể phát lại quá trình tìm kiếm.
4. Tối ưu tour: tạo pairwise matrix rồi chạy Nearest Neighbor hoặc Held-Karp.
5. Mỗi chặng đường đều dùng UCS hoặc A* tùy lựa chọn, trong khi graph ban đầu vẫn được giữ nguyên.

Các lần chạy trên cùng input sử dụng quy tắc tie-break cố định để kết quả nhất quán. Những giá trị mô phỏng trong phần thực nghiệm được ghi nhãn SIMULATED.

8 Thực nghiệm và so sánh

8.1 Benchmark tìm kiếm trên bản đồ 65 node

Nhóm em chạy benchmark trên bản đồ `thu_duc_landmarks_v1.0.0` với 10 cặp OD, một baseline scenario, 6 thuật toán và 20 lần lặp cho mỗi trường hợp. Tổng cộng có 1,200 lượt chạy được đo. Các bảng và số liệu trong phần này được lấy từ file kết quả JSON sau khi chạy benchmark.

Thuật toán	Cost trên case minh họa	Node khám phá	Nhận xét
UCS	2.606	4	Nghiệm tối ưu tham chiếu
A*	2.606	4	Nghiệm tối ưu với lower bound
BFS	2.606	4	Tối ưu chỉ khi edge cùng cost
DFS	2.606	6	Không bảo đảm tối ưu
Greedy	2.606	4	Heuristic, không bảo đảm
Bidirectional	2.606	4	Tìm kiếm từ hai đầu

Trên giao diện, thời gian xử lý thay đổi theo máy và trạng thái server nên nhóm em không dùng thời gian của một lần chụp làm kết luận chính. Cost và số node là các chỉ số ổn định hơn để so sánh thuật toán.

8.2 Kiểm tra heuristic A* với UCS

Để kiểm tra lower-bound heuristic trong nhiều điều kiện, nhóm em dùng `fixture toy_graph_v0.1` với 5 cặp OD và ba scenario: `OFFPEAK_BALANCED`, `PEAK_TRAFFIC`, `HEAVY_RAIN_SAFE`. Tổng cộng có 15 trường hợp.

Chỉ số	Kết quả
Cost mismatch A* so với UCS	0/15
Số case A* khám phá không quá UCS	15/15
Số scenario	3
Số cặp OD mỗi scenario	5

Kết quả cho thấy A* trả về cùng cost tối ưu với UCS trong cả 15 trường hợp. Số node A* khám phá cũng không lớn hơn UCS ở tất cả các case. Đây là kết quả kiểm thử cho phần cài đặt hiện tại, còn lý do về mặt lý thuyết đã được trình bày ở Chương 6.

8.3 Ảnh hưởng của cost profile

Ba profile BALANCED, PEAK_TRAFFIC và RAIN_SAFE sử dụng các hệ số được nêu ở Chương 3. Hai ví dụ trong kết quả chạy là:

- N01 → N06: baseline cost 2.343; peak cost 2.9209; rain cost 2.562625 và tuyến có thể phải đi đường vòng.
- N05 → N01: baseline cost 3.733 với path E14,E04,E02; rain cost 3.71325 với path E14,E10,E12,E02.

Các ví dụ này cho thấy khi thay đổi mức ưu tiên của traffic hoặc flood thì tuyến được chọn có thể thay đổi. Trong mọi profile, road closure vẫn được xử lý như một ràng buộc cứng.

8.4 Giải thích route và so sánh phương án thay thế

Để giải thích route rõ hơn, nhóm em không chỉ báo cáo path cuối cùng mà còn tách tổng cost theo từng cạnh. Hai ví dụ dưới đây dùng fixture `toy_graph_v0.1`; toàn bộ traffic và flood trong fixture đều mang nhãn SIMULATED. Số liệu được sinh bởi benchmark `ROUTE_EXPLANATIONS_V1`, sau đó đối chiếu lại với cost engine.

8.4.1 Case A – Peak traffic, so sánh cost thành phần

Với N01 → N06 trong scenario PEAK_TRAFFIC, UCS và A* cùng chọn N01–N02–N06, qua E01 và E03. Đây là route có composite cost thấp nhất trong các cạnh mở. Phương án thay thế đi qua E11 và E09 dài hơn nhưng quan trọng hơn là vẫn chịu cùng mức peak trong fixture. Vì fixture này đặt congestion level 4 và multiplier 1.45 cho các cạnh mở, nhóm em không kết luận rằng route đã “né kẹt xe”; kết luận đúng là profile peak làm thành phần congestion tăng và route ngắn vẫn có cost thấp hơn.

Phương án	Edge	Distance (km)	ETA (min)	Traffic	Flood	Total cost
Được chọn	E01,E03	2.500	8.062	1.125900	0.030000	2.920900
Thay thế	E01,E11,E09	3.000	11.412	1.593675	0.000000	4.011175

Các giá trị traffic và flood trong bảng cạnh dưới đây là thành phần đã nhân weight, không phải giá trị thô.

Edge	Distance	Time	Traffic	Flood	Cost
E01	1.100	2.200	0.445500	0.000000	1.160500
E03	1.400	3.360	0.680400	0.030000	1.760400
E11 (alt.)	0.800	2.670	0.540675	0.000000	1.328175
E09 (alt.)	1.100	3.000	0.607500	0.000000	1.522500

Như vậy E03 là cạnh đóng góp lớn nhất vào cost của route được chọn, chủ yếu do thời gian và congestion. Hai cạnh E11 và E09 làm phương án thay thế tăng lên 4.011175. Vì vậy route được chọn là ngắn nhất trong ví dụ này, có ETA nhỏ hơn và cũng là route có composite cost tốt nhất.

Route	Distance	Free-flow time	Traffic	Flood	Total
N01–N02–N06	0.375000	1.390000	1.125900	0.030000	2.920900
N01–N02–N04–N06	0.450000	1.967500	1.593675	0.000000	4.011175

8.4.2 Case B – Heavy rain, route đổi vì road closure

Với N05 → N01 trong scenario HEAVY_RAIN_SAFE, route baseline E14,E04,E02 có khoảng cách 3.8 km nhưng E04 bị đóng. Do đó route này không có total cost hợp lệ. UCS và A* đều chọn route thay thế E14,E10,E12,E02, tương ứng N05–N06–N04–N02–N01.

Phương án	Edge	Distance (km)	ETA (min)	Traffic	Flood	Total cost
Được chọn	E14,E10,E12,E02	4.300	14.275	0.428250	0.000000	3.713250
Route cũ	E14,E04,E02	3.800	11.388	N/A	N/A	Không hợp lệ

Các cạnh của route được chọn lần lượt đóng góp: E14 = 1.150625, E10 = 0.972500, E12 = 0.847625 và E02 = 0.742500. Bảng chi tiết là:

Edge	Distance	Time	Traffic	Flood	Cost
E14	1.300	3.550	0.133125	0.000000	1.150625
E10	1.100	3.000	0.112500	0.000000	0.972500
E12	0.800	2.670	0.100125	0.000000	0.847625
E02	1.100	2.200	0.082500	0.000000	0.742500
E04 (route cũ)	1.400	3.360	0.126000	0.500000	closed

Tổng theo component là distance 0.430000, free-flow time 2.855000, congestion 0.428250, flood 0 và tổng cost 3.713250. Route cũ bị loại không phải vì tổng cost lớn hơn mà vì E04 có flood risk 1.0 và trạng thái closed; cost engine đặt total cost của cạnh này thành null. Đây là ví dụ rõ về việc closure là ràng buộc cứng, không thể dùng weight để “bù” cho một đoạn đường đã đóng.

Route	Distance	Free-flow time	Traffic	Flood	Total
N05–N06–N04–N02–N01	0.430000	2.855000	0.428250	0.000000	3.713250
N05–N06–N02–N01	0.380000	N/A	N/A	N/A	Không hợp lệ

Trong cả hai case, UCS được dùng làm chuẩn vì tối ưu trên cost không âm. A* dùng cùng graph/scenario đã freeze, tính ρ_s một lần và dùng heuristic admissible, consistent; vì vậy A* cũng có bảo đảm tối ưu. Kết quả benchmark ghi nhận cost A* và UCS lệch 0.000000 ở cả hai case, đồng thời A* khám phá lần lượt 4 so với 5 node ở Case A và 5 so với 6 node ở Case B.

8.5 Hình minh họa kết quả chạy thuật toán

Các hình dưới đây được nhóm em chụp khi chạy ứng dụng trên máy với bản đồ 65 node. Nhóm em chỉ giữ lại một ảnh A*, một ảnh Compare và một ảnh tối ưu tour vì ba hình này đã thể hiện đủ route, trace, metrics và các chức năng chính.

9 Tối ưu tuyến giao hàng nhiều điểm

9.1 Cách áp dụng vào đồ án

Bài toán delivery gồm một depot và khoảng 5–10 điểm giao hàng. Nhóm em tính chi phí đường đi ngắn nhất giữa depot và các stop để tạo thành pairwise cost matrix. Held–Karp được dùng làm nghiệm exact reference khi số stop nhỏ; Nearest Neighbor được dùng như một heuristic nhanh để chọn stop tiếp theo có pairwise cost nhỏ nhất.

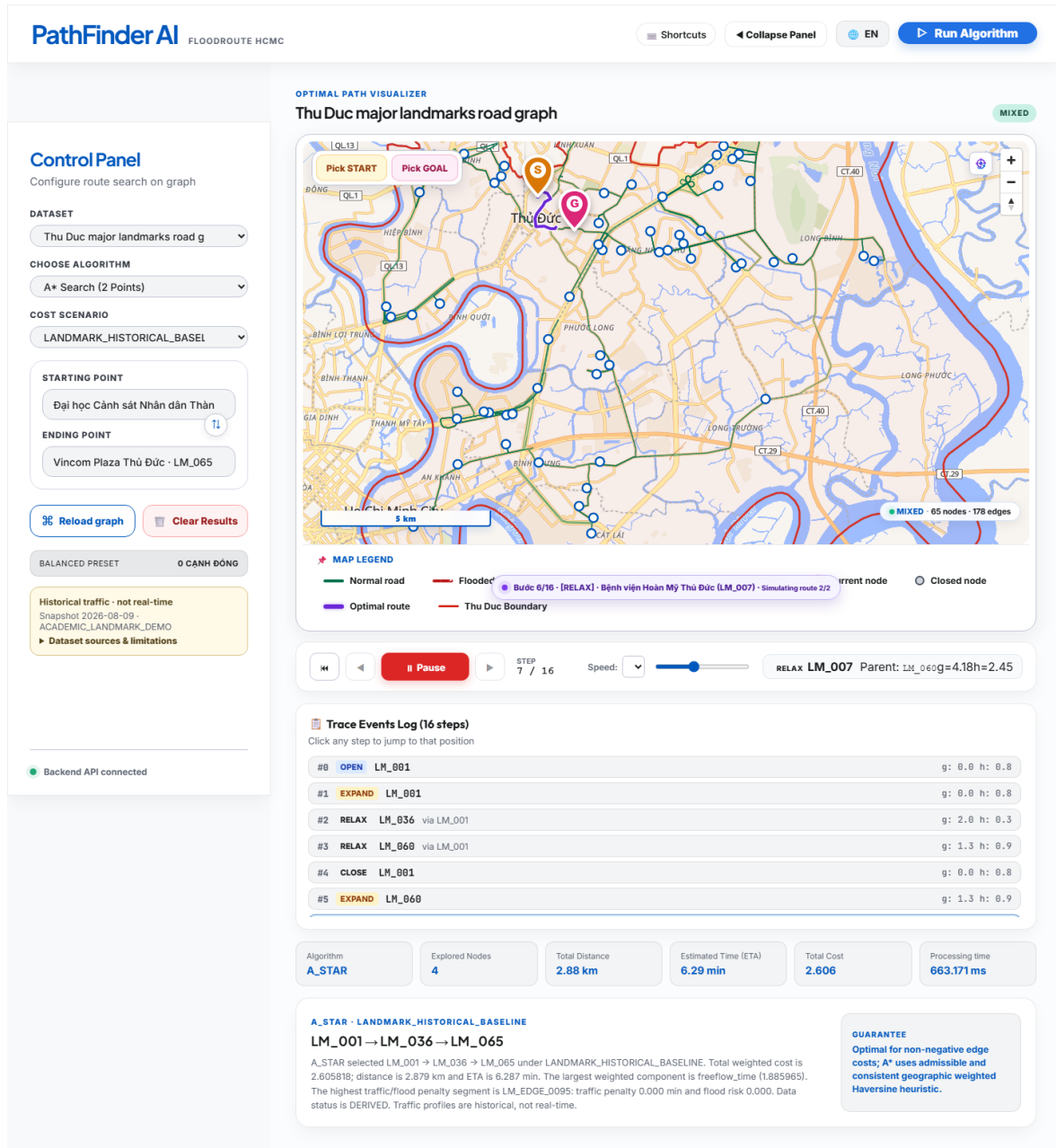


Figure 3: Kết quả chạy A* trên cặp LM_001 → LM_065. Người đọc có thể kiểm tra route, total cost khoảng 2.606, số node khám phá và guarantee tối ưu trên giao diện.

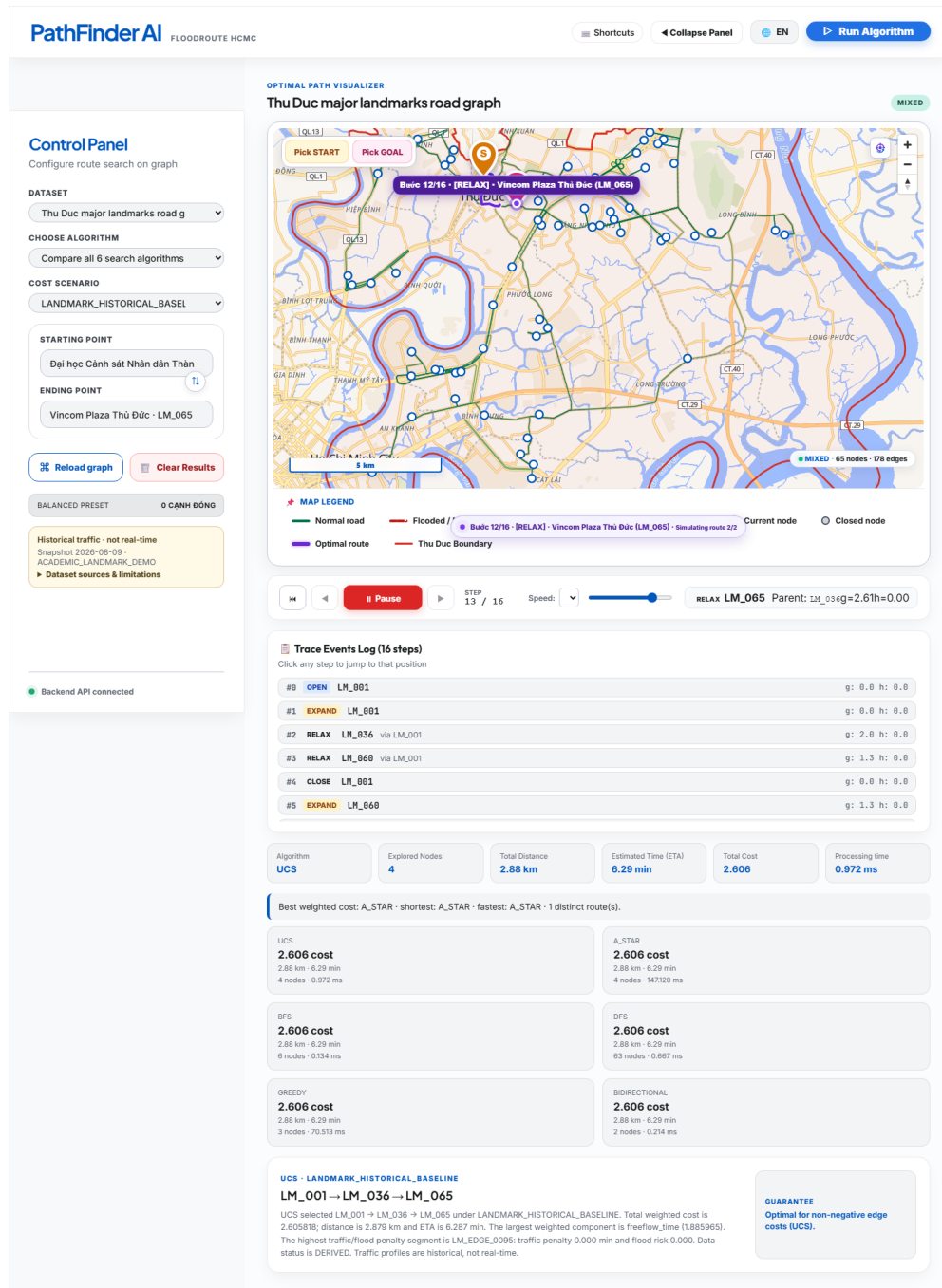
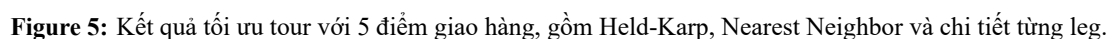


Figure 4: Chế độ Compare hiển thị cost và số node đã khám phá của 6 thuật toán, giúp thấy rõ thuật toán nào có guarantee tối ưu và thuật toán nào chỉ là heuristic.



A* hoặc UCS chỉ giải quyết từng chặng giữa hai điểm. Vì Nearest Neighbor còn phải quyết định thứ tự các stop nên việc dùng A* cho từng chặng không làm toàn bộ tour trở thành nghiệm tối ưu. Đây là lý do nhóm em so sánh riêng thứ tự ban đầu, Nearest Neighbor và Held–Karp.

9.2 So sánh kết quả tour

Phương án	Cách chọn thứ tự	Tối ưu?	Vai trò trong đồ án
Original order	Giữ thứ tự input	Không kết luận	Mốc ban đầu
Nearest Neighbor	Chọn stop rẻ nhất tiếp theo	Không	Chạy nhanh
Held–Karp	Dynamic programming trên các tập stop	Có	Nghiem tham chiếu

Hình minh họa tour ở cuối Chương 8 cho thấy giao diện đồng thời hiển thị route, tổng cost, các leg và kết quả của Held–Karp/Nearest Neighbor. Khi số stop tăng, Held–Karp tăng chi phí tính toán theo $O(n^2 2^n)$, nên Nearest Neighbor phù hợp hơn cho việc chạy nhanh trên giao diện.

10 Cài đặt, giới hạn và hướng phát triển

10.1 Cách chạy

Sau khi cài đặt theo README, nhóm em khởi động chương trình bằng:

```
npm install
npm run dev
```

Trong giao diện, người dùng chọn dataset, thuật toán, scenario, start và goal, sau đó bấm *Run Algorithm*. Với bài toán nhiều điểm, chọn depot và thêm các stop trước khi chạy phần tối ưu tour. Các lệnh kiểm thử và benchmark được giữ trong README để nhóm có thể kiểm tra lại artifact khi cần.

10.2 Giới hạn của đồ án

Dataset hiện tại chưa phải dữ liệu traffic real-time. Một phần traffic/flood trong fixture là dữ liệu SIMULATED; một số nhận định cho dữ liệu tương lai vẫn là ASSUMPTION. Nếu cho phép người dùng nhập custom weights thì cần kiểm tra giá trị đầu vào và thang đo. Held-Karp có độ phức tạp tăng theo cấp số mũ nên chỉ phù hợp với số stop nhỏ. Heuristic lower-bound bảo đảm tính tối ưu cho composite cost hiện tại, nhưng có thể không mạnh nếu cost profile thay đổi nhiều.

10.3 Hướng phát triển

Trong tương lai, nhóm em có thể bổ sung traffic real-time, thêm scenario peak/rain cho landmark release, hỗ trợ custom weights có validation, bài toán nhiều xe và time windows. Giao diện cũng có thể được mở rộng để hiển thị nhiều thông tin hơn về cost và trace. Khi thêm các thành phần mới, cần cập nhật provenance, benchmark và các contract liên quan.

10.4 Kết luận

Qua đồ án, nhóm em xây dựng được một quy trình tìm đường gồm UCS, A*, các thuật toán tìm kiếm khác và hai phương pháp tối ưu tour. UCS được dùng làm mốc để kiểm tra A*. Trên 15 trường hợp kiểm thử lower-bound, A* cho cùng cost với UCS và không khám phá nhiều node hơn UCS. Kết quả này cho thấy heuristic đang dùng phù hợp với composite cost và các scenario đã kiểm tra.

References

- [1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th Edition, Pearson, 2020.
- [2] Michael Held and Richard M. Karp. A Dynamic Programming Approach to Sequencing Problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [4] OpenStreetMap Contributors. Planet dump retrieved from <https://planet.openstreetmap.org>, 2026.
- [5] UTraffic Dataset: Urban Traffic Flow and Speed Profiling in Ho Chi Minh City. Kaggle Open Dataset, 2025.